OPERATING SYSTEMS | CS-2006

COMPREHENSIVE EXAM NOTES

Chapters 1, 2 & 3

Introduction • OS Services & System Calls • Processes & IPC

FAST NUCES | Spring 2026 | Instructor: Anaum Hamid

CHAPTER 1: Introduction to Operating Systems

1.1 What is an Operating System?

An Operating System (OS) is system software that acts as an intermediary between the user and the computer hardware. It manages hardware resources — CPU, memory, and I/O devices — on behalf of users and applications.

Core Definition
The OS is the one program running at all times — this is called the KERNEL.
Everything else is either: (a) System Program — ships with the OS but not part of the kernel, or
(b) Application Program — not associated with the OS at all.
Modern OSes also include MIDDLEWARE — software frameworks providing services like databases, multimedia, and graphics.

OS Goals

- Execute user programs and make solving user problems easier
- Make the computer convenient to use
- Use hardware resources efficiently
- Resource Allocator: Manages all resources; decides between conflicting requests for efficient/fair use
- Control Program: Controls execution of programs to prevent errors and improper use

1.2 Computer System Structure

A computer system is divided into four components:

Component	Description	Examples
Hardware	Provides basic computing resources	CPU, Memory, I/O devices
Operating System	Controls/coordinates use of hardware	Windows, Linux, macOS
Application Programs	Defines ways system resources are used	Word processor, browser, games
Users	People, machines, or other computers	Humans, servers, IoT devices

1.3 Computer-System Organization

One or more CPUs and device controllers connect through a common bus providing access to shared memory. CPUs and I/O devices execute concurrently.

Key Concepts

Device Controller: In charge of a particular device type; has a local buffer
Device Driver: OS software that manages I/O for a controller (uniform interface)
CPU moves data between main memory and controller local buffers
Device controller signals CPU completion via an INTERRUPT

1.4 Interrupts

An interrupt is a hardware or software signal that causes the CPU to stop its current task and handle the interrupt. Operating systems are fundamentally interrupt-driven.

Types of Interrupts

- Hardware Interrupt: Triggered by a device (e.g., keyboard press, I/O complete)
- Trap / Exception: Software-generated interrupt due to an error (e.g., division by zero) or a user request (system call)

Interrupt Handling Process

1. Interrupt occurs → CPU transfers control to interrupt service routine (ISR)
2. OS preserves state: saves registers and program counter
3. Interrupt vector determines which ISR to invoke
4. ISR executes to handle the interrupt
5. OS restores saved state → resumes normal execution

Interrupt Vector
The interrupt vector is a table of addresses of all interrupt service routines. Each interrupt type has a unique number that indexes into this table to find the correct handler.

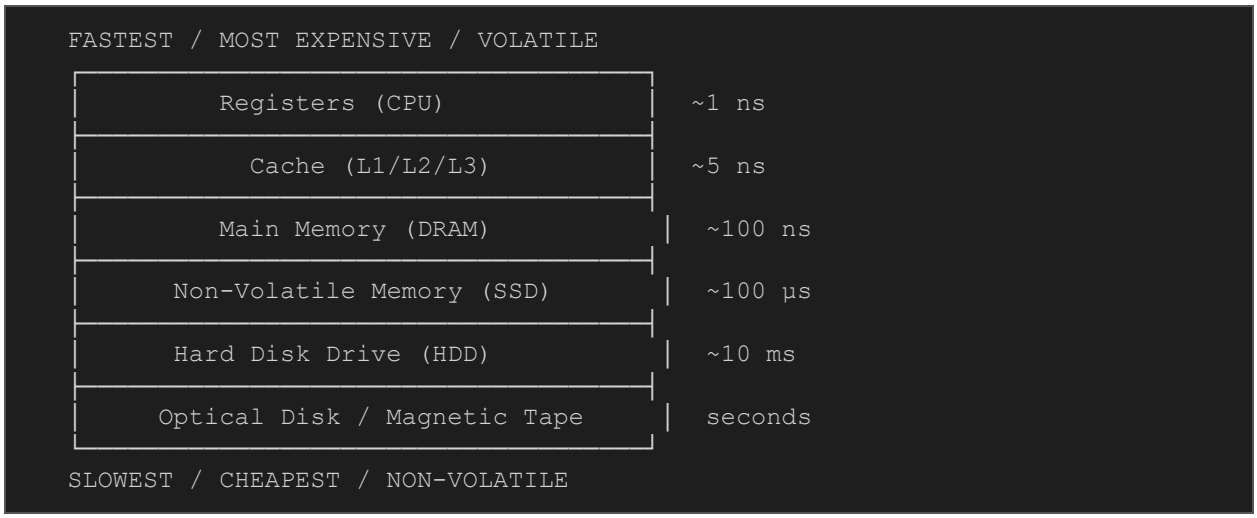
1.5 I/O Structure

Two methods for handling I/O exist:

Method	Description	Disadvantage
Synchronous I/O	Control returns to user only after I/O completes. CPU idles (wait loop).	CPU wasted while waiting; only one I/O at a time
Asynchronous I/O	Control returns immediately; OS notifies user when I/O is done.	More complex to manage
DMA (Direct Memory Access)	Device controller transfers blocks directly to memory; CPU not involved per byte.	Only one interrupt per block (efficient)

1.6 Storage Structure & Hierarchy

Storage is organized in a hierarchy based on speed, cost, and volatility:



- Main Memory: Only storage CPU can access directly; volatile; uses DRAM
- Secondary Storage: Large, non-volatile; HDD or NVM (SSD)
- Caching: Copying data into faster storage to speed access. Main memory = cache for secondary storage.

1.7 Computer System Architecture

Multiprocessor Systems (Parallel Systems)

- Increased Throughput: More work done per time unit
- Economy of Scale: Less cost than multiple single-processor systems
- Increased Reliability: Graceful degradation / fault tolerance

Type	Description	Example
Asymmetric Multiprocessing	Master processor controls; assigns tasks to slave processors	Older servers
Symmetric Multiprocessing (SMP)	All CPUs are peers; each runs OS code and user tasks	Modern PCs, servers
Multi-core	Multiple cores on a single chip — more efficient, less power	Intel Core i7
Clustered Systems	Multiple complete systems linked via SAN for high availability	Supercomputers

1.8 Operating System Operations

Bootstrap Program

Stored in ROM/EEPROM (firmware). Loaded at power-up or reboot. Initializes the system, loads the OS kernel into memory, and starts its execution.

Dual-Mode Operation

The OS protects itself and system components by distinguishing between user mode and kernel mode using a hardware MODE BIT.

Mode	Mode Bit	What Can Run	Purpose
User Mode	1	User applications	Restricted — no direct hardware access
Kernel Mode	0	OS kernel code	Privileged — full hardware access

- System Call: Switches from user mode to kernel mode. Return resets to user mode.
- Privileged instructions can only execute in kernel mode (e.g., I/O control, interrupt handling)
- Timer: Prevents infinite loops by generating an interrupt after a fixed time period; set by OS (privileged instruction)

1.9 Multiprogramming & Multitasking

Concept	Description	Key Benefit
Multiprogramming	Multiple jobs kept in memory; CPU switches when one waits (I/O)	CPU never idle; no user interaction
Multitasking (Timesharing)	CPU switches so frequently users can interact with each job	Interactive computing; response < 1 sec
Virtual Memory	Allows execution of processes not completely in memory	Run programs larger than RAM
CPU Scheduling	OS chooses which ready process runs next on CPU	Maximize CPU utilization

1.10 Kernel Data Structures

- Singly/Doubly/Circular Linked Lists: Process queues, file lists
- Binary Search Tree: O(lg n) for balanced BST — used in schedulers
- Hash Maps: O(1) lookup — used for process tables, inodes
- Bitmaps: String of n binary digits representing status of n items (e.g., free disk blocks)

Linux Examples
<linux/list.h> — Doubly linked list (used everywhere in kernel)
<linux/kfifo.h> — Circular FIFO queue
<linux/rbtree.h> — Red-black balanced BST

1.11 Computing Environments

Environment	Key Characteristics
Traditional	Stand-alone; portals for web access; home firewalls

Mobile	Smartphones/tablets; GPS, gyroscope; iOS and Android dominate; touch/voice UI
Client-Server	Clients request services; servers respond (compute-server, file-server)
Peer-to-Peer	All nodes equal — can be client OR server; examples: Skype, Napster, Gnutella
Cloud Computing	SaaS / PaaS / IaaS; public / private / hybrid; uses virtualization
Real-Time Embedded	Hard real-time constraints; special-purpose OS; most prevalent form

CHAPTER 2: Operating System Services

2.1 OS Services Overview

Operating systems provide services to programs and users. They form the link between users, applications, and hardware.

Two Categories of Services
USER-HELPFUL services: Program execution, I/O operations, file systems, communications, error detection
SYSTEM-EFFICIENCY services: Resource allocation, logging/accounting, protection and security

Service	Description
Program Execution	Load and run a program; end execution (normal or abnormal)
I/O Operations	User programs cannot perform I/O directly — OS provides controlled access
File-System Manipulation	Create, delete, read, write, search files/directories; manage permissions
Communications	Processes exchange info via shared memory or message passing (same or different systems)
Error Detection	Detect and handle CPU errors, memory errors, I/O device errors, user programs
Resource Allocation	Allocate CPU cycles, memory, files, I/O devices to multiple users/processes
Logging/Accounting	Track which users use how many resources (for billing or statistics)
Protection & Security	Ensure all access to resources is controlled; authenticate users

2.2 User and OS Interfaces

Command-Line Interface (CLI) / Shell

- Allows users to type commands directly (e.g., bash, PowerShell)
- Two approaches: Commands built into shell, or fetched from file system
- Powerful for scripting and automation

Graphical User Interface (GUI)

- Desktop metaphor: icons, windows, mouse, keyboard
- Examples: Windows Aero, macOS Aqua, GNOME/KDE on Linux

Touch-Screen Interface

- No mouse; gestures replace commands (tap, swipe, pinch)
- Used in smartphones (iOS, Android)

2.3 System Calls

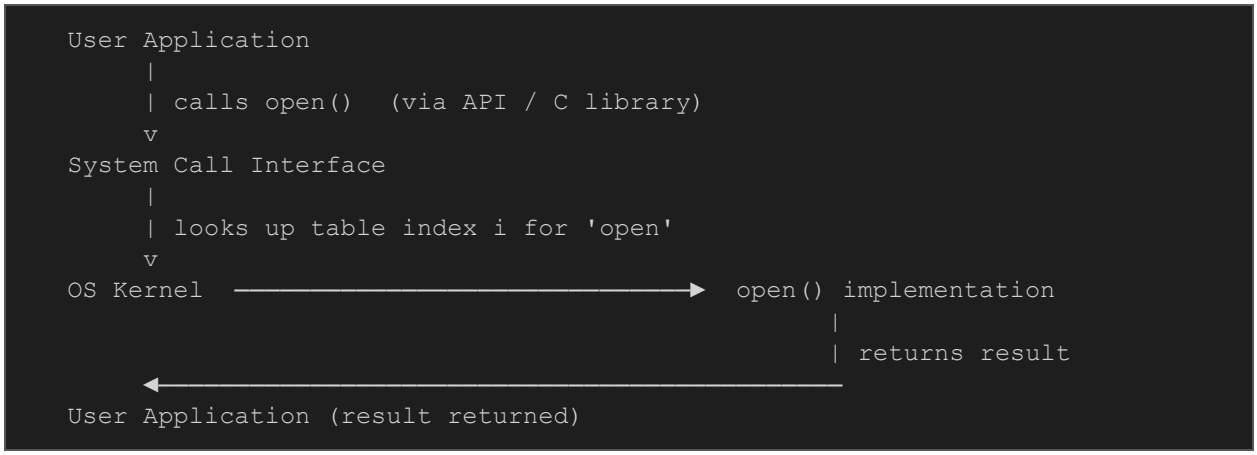
System calls are the programming interface between a running program and the operating system. They provide the mechanism for a user program to request OS services.

- Written in C or C++ (low-level assembly for some)
- Programs access system calls through an Application Programming Interface (API) — not directly
- Common APIs: Win32 API (Windows), POSIX API (Unix/Linux/macOS), Java API (JVM)

> **Why use APIs instead of direct system calls?**
> Portability: Same API code runs on any system that supports that API
> Simplicity: API hides complexity of system call implementation
> Safety: API manages the system-call interface and run-time support library

2.4 System Call Implementation

Each system call has a unique NUMBER. The system-call interface maintains an indexed table (system call table) keyed by these numbers.



2.5 System Call Parameter Passing

Three methods for passing parameters to the OS:

Method	Description	Limitation
Registers	Pass parameters directly in CPU registers	Limited by number of registers
Block/Table	Store parameters in a memory block; pass address in register	No limit on count/length (used by Linux, Solaris)

Stack	Program pushes parameters; OS pops them off the stack	No limit on count/length

2.6 Types of System Calls

Category	Examples (Unix / Windows)
Process Control	fork()/CreateProcess(), exit()/ExitProcess(), wait()/WaitForSingleObject()
File Management	open()/CreateFile(), read()/ReadFile(), write()/WriteFile(), close()/CloseHandle()
Device Management	ioctl()/SetConsoleMode(), read()/ReadConsole(), write()/WriteConsole()
Information Maintenance	getpid()/GetCurrentProcessID(), alarm()/SetTimer(), sleep()/Sleep()
Communications	pipe()/CreatePipe(), shm_open()/CreateFileMapping(), mmap()/MapViewOfFile()
Protection	chmod()/SetFileSecurity(), umask()/InitializeSecurityDescriptor()

2.7 System Call Code Examples

Example 1: File Copy using System Calls (C / Linux)

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    int src_fd = open(argv[1], O_RDONLY);          // system call: open()
    if (src_fd == -1) { perror("open src"); exit(1); }

    int dest_fd = open(argv[2], O_WRONLY|O_CREAT|O_TRUNC, 0644);
    if (dest_fd == -1) { perror("open dest"); exit(1); }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0) {
        bytes_written = write(dest_fd, buffer, bytes_read);
        if (bytes_written != bytes_read) { perror("write"); exit(1); }
    }

    close(src_fd);
    close(dest_fd);
    return 0;
}
```

Code Walkthrough
open() → File Management system call; returns file descriptor (int fd)
read() → Fills a user-space buffer from the file; returns bytes actually read
write() → Transfers data from buffer to file; returns bytes written
close() → Releases the file descriptor back to the OS
Each of these triggers a mode switch: User Mode → Kernel Mode → back to User Mode

Example 2: Process Creation using fork() (C / Linux)

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();    // system call: fork() — creates child process

    if (pid < 0) {         // fork() FAILED
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {   // CHILD process
        printf("Child: PID=%d, Parent PID=%d\n", getpid(), getppid());
        execlp("/bin/ls", "ls", NULL);
        perror("exec failed");
    }
    else {                 // PARENT process
        printf("Parent: PID=%d, Child PID=%d\n", getpid(), pid);
        wait(NULL);
        printf("Child has terminated.\n");
    }
    return 0;
}
```

fork() Return Values — Critical for Exams
fork() returns < 0 → ERROR: child not created
fork() returns == 0 → You are in the CHILD process
fork() returns > 0 → You are in the PARENT process; value = child's PID
After fork(), both parent and child run the same code below fork() — but in separate address spaces

Example 3: POSIX Shared Memory (IPC)

```c
// --- PRODUCER (Writer) ---
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/mman.h>

int main() {
    int shm_fd = shm_open("/myshm", O_CREAT | O_RDWR, 0666);
```

```
    ftruncate(shm_fd, 4096);
    char *ptr = mmap(0, 4096, PROT_WRITE, MAP_SHARED, shm_fd, 0);
    sprintf(ptr, "Hello from Producer!");
    return 0;
}

// --- CONSUMER (Reader) ---
int main() {
    int shm_fd = shm_open("/myshm", O_RDONLY, 0666);
    char *ptr = mmap(0, 4096, PROT_READ, MAP_SHARED, shm_fd, 0);
    printf("Read: %s\n", ptr);
    shm_unlink("/myshm");
    return 0;
}
```

2.8 Linkers and Loaders

The journey from source code to a running program involves several steps:

```
  Source Code (main.c)
        |
        | gcc -c main.c            [COMPILE]
        v
  Object File (main.o)  <-- other .o files
        |
        | gcc -o main main.o -lm   [LINK]
        v
  Executable (main)  <-- dynamically linked libraries (.so / .dll)
        |
        | ./main                   [LOAD & EXECUTE]
        v
  Program in Memory (loaded by OS loader)
```

Stage	Tool	Description
Compilation	Compiler (gcc -c)	Source → relocatable object file (.o); can be loaded anywhere in memory
Linking	Linker (gcc -o)	Combines .o files + libraries → single binary executable
Loading	Loader (OS)	Loads binary into memory; performs relocation (assigns final addresses)
Relocation	Loader	Adjusts code and data to match actual memory addresses

Library Type	Description	Extension
Static Library	Copied into executable at link time (larger executable, self-contained)	Linux: .a Windows: .lib
Dynamic Library	Linked at runtime; shared in memory by all processes using it	Linux: .so Windows: .dll

Executable File Formats
ELF (Executable and Linkable Format) — Linux and UNIX systems
PE (Portable Executable Format) — Windows systems
Mach-O Format — macOS systems

2.9 OS Structure Designs

Structure	Description	Performance	Security	Example
Monolithic	All OS in one large binary in kernel mode	High	Low (a bug crashes system)	Linux (old), MS-DOS, Unix
Layered	OS divided into hierarchy of layers; each uses only lower layers	Moderate	Moderate	THE OS, MULTICS
Microkernel	Minimal kernel; services in user space; IPC via message passing	Lower (IPC overhead)	Very High	Mach, QNX, macOS (partial)
Modular	Core kernel + dynamically loadable modules (LKMs)	High	High	Linux (modern), Solaris
Hybrid	Combines approaches (e.g., microkernel + modules)	Good	Good	Windows NT, macOS/iOS

2.10 System Boot

The boot process loads the OS kernel into memory and starts execution:

```
Power ON
   |
   v
BIOS / UEFI -- POST (Power-On Self Test); detects hardware
   |
   v
Bootloader (GRUB2 / LILO) -- loaded from MBR/EFI partition
   | -- presents boot menu; user selects OS
   v
Kernel Initialization -- decompressed; hardware drivers loaded
   | -- mounts initramfs; starts essential services
   v
init / systemd -- initializes all system services; mounts filesystems
   |
   v
Login Prompt  (tty for CLI, display manager for GUI)
```

BIOS vs UEFI
BIOS: Legacy firmware; 16-bit; uses MBR (max 2 TB disk); no GUI

UEFI: Modern replacement; 32/64-bit; uses GPT (>2 TB); GUI support; Secure Boot; faster POST

CHAPTER 3: Processes

3.1 Process Concept

A PROCESS is a program in execution. It is an active entity. A PROGRAM is a passive entity stored on disk. When a program is loaded into memory and started, it becomes a process.

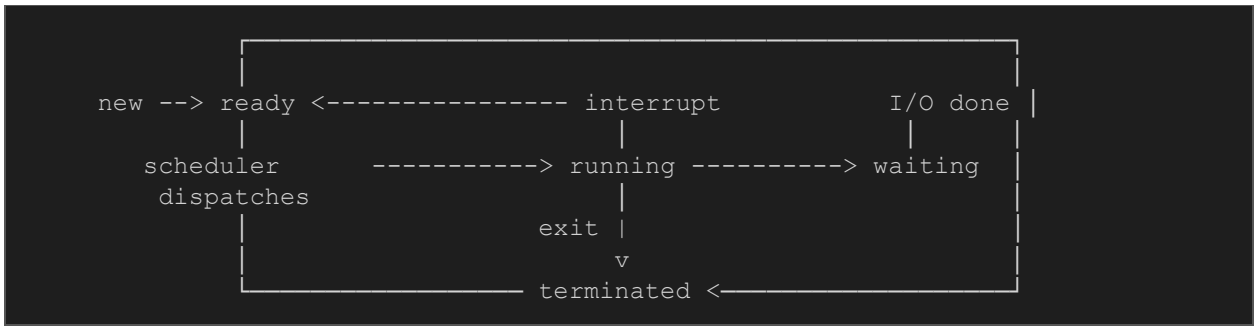
Memory Layout of a Process

> **One Program → Multiple Processes**
> Multiple users running the same program (e.g., multiple Firefox instances) each create a separate process. Each process has its own address space (stack, heap, data) but can share the text (code) section.

3.2 Process States

As a process executes, it changes state. A process can be in exactly ONE state at any given time:

State	Description
New	Process is being created (PCB allocated, resources assigned)
Ready	Process is in memory, waiting to be assigned to a CPU core
Running	Instructions are being executed on a CPU core
Waiting (Blocked)	Process is waiting for an event (e.g., I/O completion, signal)
Terminated	Process has finished execution; resources being deallocated



3.3 Process Control Block (PCB)

The PCB (also called Task Control Block) is the data structure the OS uses to represent and manage a process. Each process has exactly one PCB.

PCB Field	Description
Process State	Current state: running, ready, waiting, new, terminated
Program Counter	Address of next instruction to execute
CPU Registers	All process-specific register contents (saved on context switch)
CPU Scheduling Info	Priority, scheduling queue pointers
Memory Management Info	Base/limit registers, page tables, segment tables
Accounting Info	CPU time used, clock time since start, time limits
I/O Status Info	List of I/O devices allocated, list of open files

PCB in Linux (task_struct)

```
struct task_struct {
    pid_t              pid;       /* process identifier */
    long               state;     /* state of the process */
    unsigned int       time_slice;/* scheduling information */
    struct task_struct *parent;   /* this process's parent */
    struct list_head   children;  /* this process's children */
    struct files_struct *files;   /* list of open files */
    struct mm_struct   *mm;       /* address space (memory) */
};
```

3.4 Process Scheduling

The Process Scheduler selects which ready process to run next on the CPU. Goal: Maximize CPU utilization and quickly switch processes.

Scheduling Queues

- Ready Queue: Set of all processes in main memory, ready and waiting to execute

- Wait Queue(s): Set of processes waiting for a specific event (one queue per event type)
- Processes migrate among queues as their state changes

Context Switch

When the CPU switches from one process to another, the OS must:

6. Save the state of the old process into its PCB (registers, PC, memory maps)
7. Load the saved state of the new process from its PCB
8. Resume execution of new process where it left off

> **Warning: Context Switch Overhead**
> Context-switch time is PURE OVERHEAD — no useful work is done during the switch
> The more complex the OS/PCB, the longer the context switch takes
> Hardware support (multiple register sets) can reduce context-switch time

3.5 Operations on Processes

Process Creation

Parent processes create child processes, forming a TREE of processes. Each process is identified by a unique PID (Process Identifier).

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();   // STEP 1: Create child

    if (pid == 0) {       // CHILD
        execlp("/bin/ls", "ls", "-l", NULL);  // STEP 2: Load new program
    } else {              // PARENT
        wait(NULL);       // STEP 3: Wait for child to finish
        printf("Done\n");
    }
    return 0;
}
```

System Call	Purpose	Return Value
fork()	Creates a duplicate child process	0 to child, child's PID to parent, -1 on error
exec()	Replaces process memory with new program	Only returns on error
wait()	Parent waits for child to terminate	PID of terminated child
exit()	Terminates calling process, returns status	No return (process ends)
getpid()	Returns PID of calling process	Current process PID
getppid()	Returns PID of parent process	Parent process PID

Process Termination

- Normal termination: last statement executed, then exit() system call
- Parent can terminate child using abort() — reasons: child exceeded resources, task no longer needed, parent is exiting
- Cascading Termination: If parent terminates, all children must be terminated (OS-initiated)

Term	Definition
Zombie	Child has terminated, but parent hasn't called wait() yet. PCB still exists.
Orphan	Parent terminated without calling wait(). Child has no parent.

3.6 Interprocess Communication (IPC)

Cooperating processes need to exchange data. Two fundamental models exist:

Model	How it works	Advantage	Disadvantage
Shared Memory	Both processes map the same memory region; read/write via pointers	Fast (no system calls after setup)	Requires synchronization (race conditions)
Message Passing	OS provides send()/receive() primitives; OS manages the channel	No synchronization needed; easy to implement	Slower (every exchange requires system call)

3.7 Shared Memory

The Producer-Consumer problem is the classic paradigm for shared memory IPC:

Bounded-Buffer (Shared Memory) Implementation

```
#define BUFFER_SIZE 10

typedef struct { /* item type */ } item;

item buffer[BUFFER_SIZE];  // shared buffer
int  in  = 0;              // next free position
int  out = 0;              // first full position

// PRODUCER
while (true) {
    while (((in + 1) % BUFFER_SIZE) == out) ; // wait if FULL
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}

// CONSUMER
while (true) {
```

```
    while (in == out) ; // wait if EMPTY
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

Buffer States
EMPTY condition: in == out
FULL condition: ((in + 1) % BUFFER_SIZE) == out
Note: This implementation can only hold BUFFER_SIZE - 1 items (one slot always wasted)

3.8 Message Passing

Message passing requires at least two operations: send(message) and receive(message).

Design Decisions for Message Passing

Issue	Options
Naming (Direct vs Indirect)	Direct: send(P, msg) — name recipient explicitly. Indirect: send(mailbox_A, msg) — use mailbox/port
Synchronization	Blocking (Synchronous): sender waits until receiver gets message. Non-blocking (Async): sender continues immediately
Buffering	Zero capacity (no buffer, must rendezvous), Bounded capacity (finite queue), Unbounded capacity (infinite queue)

3.9 Pipes

A pipe is one of the oldest and simplest IPC mechanisms. It acts as a conduit between two processes.

Feature	Ordinary (Anonymous) Pipe	Named Pipe (FIFO)
Relationship	Requires parent-child relationship	No parent-child relationship needed
Direction	Unidirectional (write-end to read-end)	Bidirectional
Scope	Only within same process family	Any process with permissions
Persistence	Destroyed when processes terminate	Exist as file in filesystem
Unix Name	pipe()	mkfifo() / named FIFO

Ordinary Pipe — Code Example

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define BUFFER_SIZE 25
```

```
#define READ_END  0
#define WRITE_END 1

int main() {
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    int fd[2];

    pipe(fd);       // Create pipe: fd[0]=read end, fd[1]=write end
    pid_t pid = fork();

    if (pid == 0) {                    // CHILD -- reads from pipe
        close(fd[WRITE_END]);
        read(fd[READ_END], read_msg, BUFFER_SIZE);
        printf("Child read: %s\n", read_msg);
        close(fd[READ_END]);
    } else {                           // PARENT -- writes to pipe
        close(fd[READ_END]);
        write(fd[WRITE_END], write_msg, strlen(write_msg)+1);
        close(fd[WRITE_END]);
        wait(NULL);
    }
    return 0;
}
```

3.10 Chrome Browser — Multi-Process Architecture

A famous real-world example of process architecture:

Process Type	Responsibility
Browser Process	Manages user interface, disk and network I/O — one per browser
Renderer Process	Renders web pages (HTML, CSS, JS). One per website tab. Runs in a SANDBOX.
Plugin Process	One per plugin type (e.g., Flash, PDF viewer)

Why Multi-Process?
If one renderer process crashes (bad website), only that tab is affected — not the whole browser
Sandbox: Renderer processes have restricted disk and network I/O (security isolation)
Old single-process browsers: one bad site causes the entire browser to hang or crash

EXAM PRACTICE QUESTIONS

SECTION A: Chapter 1 — Theory Questions

Q1. What is an Operating System? Distinguish between an OS as a resource allocator vs. control program.

Model Answer
An OS is system software that acts as an intermediary between users/applications and hardware.
As a RESOURCE ALLOCATOR: The OS manages all hardware resources (CPU, memory, I/O devices) and decides how to allocate them fairly and efficiently among competing requests.
As a CONTROL PROGRAM: The OS controls the execution of user programs to prevent errors and improper use of the computer (e.g., unauthorized memory access).

Q2. Explain the purpose and mechanism of interrupts in an OS. Differentiate between hardware interrupts and traps.

Model Answer
An interrupt causes the CPU to stop its current task and transfer control to an Interrupt Service Routine (ISR) via the interrupt vector table.
Mechanism: (1) Interrupt signal raised -> (2) CPU saves state -> (3) Interrupt vector consulted -> (4) ISR executes -> (5) State restored, execution resumes
Hardware Interrupt: Generated by external device (e.g., keyboard, I/O controller). Asynchronous.
Trap/Exception: Software-generated interrupt. Either an ERROR (e.g., division by zero) or a deliberate user REQUEST (system call). Synchronous.
The OS is fundamentally interrupt-driven — it responds to events through interrupts.

Q3. Explain dual-mode operation. Why is it important for OS security?

Model Answer
The hardware provides a MODE BIT: 0 = Kernel Mode, 1 = User Mode.
USER MODE: User applications run here; limited — cannot execute privileged instructions or directly access hardware.
KERNEL MODE: OS kernel runs here; full access to hardware and all machine instructions.
Mode Switch: A system call switches from user to kernel mode. When the OS returns, mode reverts to user.
Importance: Prevents user programs from accidentally or maliciously interfering with the OS or other programs.

Q4. Compare multiprogramming and multitasking (timesharing). What OS mechanisms does each require?

Model Answer
MULTIPROGRAMMING: Multiple jobs kept in memory simultaneously. When one job waits (for I/O), OS switches to another. Goal: keep CPU busy. Non-interactive (batch). Requires: job scheduling, memory management.
MULTITASKING: Extension of multiprogramming where CPU switches so frequently users can INTERACT with each running job. Goal: response time < 1 second. Requires: CPU scheduling, virtual memory, swapping.
Key difference: Multiprogramming optimizes CPU utilization; multitasking provides interactive user experience.

SECTION B: Chapter 2 — Theory Questions

Q5. What is a system call? Why do programs use APIs rather than direct system calls?

Model Answer
A system call is the programming interface between a running user process and the OS kernel. It allows a process to request OS services (file I/O, process creation, etc.).
Programs use APIs because: (1) PORTABILITY — the same API code works on any system supporting that API; (2) SIMPLICITY — API hides low-level details; (3) SAFETY — the run-time library manages the interface correctly.
Examples: POSIX API (Unix/Linux/macOS), Win32 API (Windows), Java API (JVM).

Q6. Describe three methods for passing parameters to system calls. Which does Linux use?

Model Answer
1. REGISTERS: Parameters passed directly in CPU registers. Simplest method. Limited by number of registers.
2. BLOCK/TABLE: Parameters stored in a memory block; the address of the block is passed in a register. No limit on count or length. Used by LINUX and Solaris.
3. STACK: Parameters pushed onto the stack by the program, popped off by the OS. No limit on count or length.
Linux uses the BLOCK/TABLE method.

Q7. Compare monolithic kernel, microkernel, and modular OS structures.

Model Answer
MONOLITHIC: All OS functions in one large binary running in kernel mode. Fast (no mode switches between components) but hard to modify and a bug anywhere can crash the system. Example: Linux (older), MS-DOS.
MICROKERNEL: Only core functions in kernel (IPC, memory management, basic scheduling). Everything else in user space via message passing. More secure and maintainable but slower due to IPC overhead. Example: Mach, QNX.
MODULAR: Core kernel + Loadable Kernel Modules (LKMs) added at runtime. Flexibility of microkernel with performance of monolithic. Example: Linux (modern), Solaris.
Exam tip: Linux is BOTH monolithic AND modular (monolithic plus modular design).

SECTION C: Chapter 3 — Theory Questions

Q8. What is a process? How does it differ from a program? What are the five process states?

Model Answer

A PROGRAM is a passive entity stored on disk (executable file). A PROCESS is an active entity — a program loaded into memory and in execution.

Difference: One program can create multiple processes. A process includes program code PLUS its current state (registers, stack, heap, data).

Five states: (1) NEW — being created; (2) READY — waiting for CPU; (3) RUNNING — executing; (4) WAITING/BLOCKED — waiting for event; (5) TERMINATED — finished execution.

Q9. What is a PCB? List and explain its fields.

Model Answer

The Process Control Block (PCB) is the OS data structure that represents a process. Each process has exactly one PCB.

Fields: (1) Process State; (2) Program Counter — next instruction address; (3) CPU Registers — saved register values; (4) Scheduling Info — priority, queue pointers; (5) Memory Info — page/segment tables; (6) Accounting Info — CPU time used; (7) I/O Status — open files, allocated devices.

The PCB is the most important data structure in the OS — without it, context switching and process management would be impossible.

Q10. Explain context switching. Why is it called 'pure overhead'?

Model Answer

A context switch occurs when the CPU switches from executing one process to another.

Process: (1) Save current process state (all registers, PC, memory mappings) into its PCB; (2) Load new process state from its PCB; (3) Resume new process.

Pure overhead: During the context switch, the CPU is not executing any useful user or OS code — it is only saving and restoring state. The system makes no progress on any user task during this time.

Reducing overhead: Hardware multiple register sets; minimize context switches through better scheduling.

Q11. Compare shared memory and message passing IPC. Give an example use case for each.

Model Answer

SHARED MEMORY: Both processes map the same memory region; communicate by reading/writing. Fast — after initial setup, no system calls needed. But requires explicit synchronization (semaphores, mutex) to avoid race conditions. Use case: High-performance data exchange (e.g., video processing pipeline).

MESSAGE PASSING: OS provides send()/receive() primitives. No synchronization needed — OS manages the channel. Slower (each message requires system calls). Use case: Network communication between processes on different machines.
Key tradeoff: Shared memory is faster but more complex; message passing is simpler but slower.

Q12. What are zombie and orphan processes? How does the OS handle them?

Model Answer

ZOMBIE: A process that has terminated but whose parent has NOT yet called wait(). The PCB still exists so the parent can retrieve the exit status. The zombie occupies no CPU/memory resources but does occupy a PID and a PCB entry.
ORPHAN: A process whose parent has terminated WITHOUT calling wait(). The orphan still runs but has no parent to collect its exit status.
Handling: In Linux/Unix, orphans are re-parented to the 'init' process (PID 1 / systemd), which periodically calls wait() to clean them up.

NUMERICAL PRACTICE QUESTIONS

Numerical 1: fork() Process Tree

QUESTION
How many processes are created by the following code snippet? fork(); fork(); fork();

SOLUTION
Each fork() doubles the number of processes.
Start: 1 process (P0)
After fork 1: 2 processes (P0, P1)
After fork 2: 4 processes (P0, P1, P2, P3)
After fork 3: 8 processes (P0 through P7)
Formula: 2^n where n = number of fork() calls -> 2^3 = 8 processes total
Note: The ORIGINAL process counts. Total NEW processes created = 8 - 1 = 7

Numerical 2: fork() Output Prediction

QUESTION
What is the output of the following program?

```
int main() {
    int x = 10;
    pid_t pid = fork();
    if (pid == 0) {
        x = 20;
        printf("Child: x = %d\n", x);
    } else {
        x = 30;
        printf("Parent: x = %d\n", x);
    }
    printf("Final x = %d\n", x);
    return 0;
}
```

SOLUTION
After fork(), child gets a COPY of the parent's address space (x = 10).
Child executes: x = 20 -> prints 'Child: x = 20' -> prints 'Final x = 20'
Parent executes: x = 30 -> prints 'Parent: x = 30' -> prints 'Final x = 30'
Total output (order of Child vs Parent may vary):
Child: x = 20
Final x = 20

Parent: x = 30
Final x = 30
Key: fork() uses COPY-ON-WRITE. The child's x change does NOT affect the parent's x.

Numerical 3: Bounded Buffer State Tracking

QUESTION
A bounded buffer has BUFFER_SIZE = 5. Initial state: in = 0, out = 0 (empty).
Trace the following operations:
1. Producer produces item A
2. Producer produces item B
3. Consumer consumes one item
4. Producer produces items C, D, E
5. Producer tries to produce item F — what happens?

SOLUTION
Initial: in=0, out=0 (EMPTY: in == out)

1. Produce A: buffer[0]=A, in=(0+1)%5=1 -> State: in=1, out=0
2. Produce B: buffer[1]=B, in=(1+1)%5=2 -> State: in=2, out=0
3. Consume: out=(0+1)%5=1 (removes A) -> State: in=2, out=1
4. Produce C: buffer[2]=C, in=3
   Produce D: buffer[3]=D, in=4
   Produce E: buffer[4]=E, in=(4+1)%5=0 -> State: in=0, out=1

5. Try F: Check FULL: ((in+1)%BUFFER_SIZE)==out -> ((0+1)%5)==1 -> 1==1 -> TRUE
   Buffer is FULL! Producer WAITS (spin loop).

Buffer holds BUFFER_SIZE-1 = 4 items max (one slot wasted to distinguish full/empty)

Numerical 4: System Call Sequence Trace

QUESTION
Trace the system calls and mode transitions when a user program calls printf("Hello"):

SOLUTION
User program calls printf("Hello") -> User Mode
  |
C library (glibc) intercepts printf() -> still User Mode
  | [TRAP / SYSTEM CALL instruction]
Mode bit: 1->0 (User Mode -> Kernel Mode)
  |
System call table indexed to find write() implementation

|
OS kernel executes write() -> writes to stdout (file descriptor 1)
| [return from system call]
Mode bit: 0->1 (Kernel Mode -> User Mode)
|
Control returns to C library, then to user program

Summary: printf() -> [glibc] -> write() system call -> [kernel] -> output -> return

Numerical 5: Process State Transitions

QUESTION
A process P goes through the following sequence of events:
a) P is admitted to the system
b) The scheduler selects P to run
c) P issues a disk read request
d) The disk read completes
e) The scheduler selects another process Q; P is not chosen
f) The scheduler eventually selects P again
g) P completes and calls exit()
Trace the state of P at each step.

SOLUTION
a) P admitted -> State: NEW -> READY
b) Scheduler selects P -> State: READY -> RUNNING
c) P issues disk read -> State: RUNNING -> WAITING (blocked on I/O)
d) Disk read completes -> State: WAITING -> READY
e) Scheduler picks Q -> State: READY (P waits in ready queue)
f) Scheduler selects P -> State: READY -> RUNNING
g) P calls exit() -> State: RUNNING -> TERMINATED

State sequence: NEW -> READY -> RUNNING -> WAITING -> READY -> RUNNING -> TERMINATED

Numerical 6: Linker/Loader Addresses

QUESTION
A program has three compiled object files with the following sizes:
module_A.o: 200 bytes (code: 100, data: 100)
module_B.o: 150 bytes (code: 100, data: 50)
module_C.o: 250 bytes (code: 150, data: 100)
The OS loader places the program starting at address 3000 (base).
(a) What is the total size of the executable?
(b) If module_A originally references symbol X at offset 50, what is its final address?

SOLUTION
(a) Total size = 200 + 150 + 250 = 600 bytes

(b) Linker layout (all code sections first, then data):
Code: module_A (0-99) | module_B (100-199) | module_C (200-349)
Data: module_A (350-449) | module_B (450-499) | module_C (500-599)

Symbol X at offset 50 in module_A's CODE section:
Linker offset = 0 (module_A code starts at offset 0 in executable)
Offset 50 in module_A = offset 50 in executable
Final virtual address = base + offset = 3000 + 50 = 3050

This is RELOCATION — the loader adjusts addresses from link-time to runtime.

Quick Revision — Key Facts for Exam

Topic	Key Fact
OS Definition	Program running at all times = KERNEL; rest = system/application programs
Interrupt Vector	Table of ISR addresses; interrupt number indexes into it
DMA	One interrupt per BLOCK (not per byte); CPU not involved during transfer
Dual-Mode Bit	0 = Kernel mode, 1 = User mode; system call switches 1->0; return 0->1
Context Switch	Pure overhead; saves/restores PCB; no useful work done during switch
fork() Returns	<0 = error, ==0 = child process, >0 = parent process (value = child PID)
Zombie	Terminated child whose parent hasn't called wait() yet
Orphan	Process whose parent died without calling wait()
Shared Memory IPC	Fastest; needs synchronization; shm_open()+mmap()
Message Passing	Simpler; OS manages channel; send()/receive()
Pipe Direction	Ordinary pipe = UNIDIRECTIONAL. Named pipe (FIFO) = BIDIRECTIONAL
Buffer Full	((in+1) % BUFFER_SIZE) == out
Buffer Empty	in == out
n fork() calls	Creates 2^n total processes
Monolithic + Modular	Linux uses BOTH (monolithic base + loadable kernel modules)
ELF / PE / Mach-O	Linux / Windows / macOS executable formats
UEFI	Replaces BIOS; supports GPT, Secure Boot, >2TB drives

— End of Notes —

CS-2006 Operating Systems | FAST NUCES Spring 2026